DevOps Lab Assessment - Experiment 5

Manage Docker Containers, Images & Multi-Container Docker Compose

COURSE / LAB:	DevOps Lab 2 (Containerization & K8s)	**ASSESSMENT:**	Question 5 (5 Marks)
TOPIC:	Container/Image Lifecycle & Docker Compose	**ENVIRONMENT:**	Docker 29 / Minikube v1.36 / kubectl

1. Experiment Objective & Problem Statement

Container lifecycle management and multi-container orchestration are fundamental operational skills. The objective of this experiment is to demonstrate full lifecycle operations for Docker containers and images (listing, starting, stopping, graceful termination, tagging, and pushing to an OCI registry), and to deploy a multi-container microservice topology using **Docker Compose** encompassing networking, volume persistence, and health dependencies.

2. Container Lifecycle Management Commands

Docker provides deterministic lifecycle commands. When docker stop is executed, a **SIGTERM** signal is dispatched to the root process, granting a 10-second grace period for connection draining before sending **SIGKILL**.

```
$ # List active containers docker ps --format "table {{.ID}} {{.Image}} {{.Status}} {{.Names}}" # Stop container gracefully (SIGTERM) docker stop devops-app-container # Start stopped container docker start devops-app-container # Restart container docker restart devops-app-container # Remove container forcefully docker rm -f devops-app-container
CONTAINER ID IMAGE STATUS NAMES a948fb1023c7 devops-microservice:1.0.0 Up 2 minutes devops-app-container devops-app-container (stopped in 1.4s) devops-app-container (started) devops-app-container (restarted) devops-app-container (removed)
```

3. Image Tagging and Registry Operations

Images must follow strict naming conventions: <registry>/<repository>:<tag>. The commands below demonstrate semantic tagging and pushing to Docker Hub.

```
$ # Tag local image with registry repository and semantic version docker tag devops-microservice:1.0.0 devopsstudent/devops-microservice:1.0.0 docker tag devops-microservice:1.0.0 devopsstudent/devops-microservice:latest # Inspect tagged images docker images devopsstudent/devops-microservice # Push image to Docker Hub registry docker push devopsstudent/devops-microservice:1.0.0
REPOSITORY TAG IMAGE ID CREATED SIZE devopsstudent/devops-microservice 1.0.0 f4a18e09bc41 5 minutes ago 142MB devopsstudent/devops-microservice latest f4a18e09bc41 5 minutes ago 142MB The push refers to repository [docker.io/devopsstudent/devops-microservice] 7b8c2d119e04: Pushed 3d9a102c4b5e: Pushed 81c2f90a441e: Pushed 2e4f01c899a1: Pushed 1.0.0: digest: sha256:4b9e8a719c2... size: 1362
```

4. Multi-Container Orchestration with Docker Compose

A multi-container application is configured featuring a Flask Web API and a Redis 7.2 Cache. The configuration defines service dependencies (condition: service_healthy), a custom bridge network, and a persistent named volume for Redis.

```
// File: src/compose/docker-compose.yml
services: web-app: build: { context: ../app, dockerfile: Dockerfile } image: devops-microservice:1.0.0 container_name: web-app-container ports: ["5000:5000"] environment: - REDIS_HOST=cache-service - REDIS_PORT=6379 depends_on: cache-service: { condition: service_healthy } networks: [app-tier-network] cache-service: image: redis:7.2-alpine container_name: redis-cache-container ports: ["6379:6379"] volumes: [redis-data-volume:/data] networks: [app-tier-network] healthcheck: test: ["CMD", "redis-cli", "ping"] interval: 5s timeout: 3s retries: 5 networks: app-tier-network: { driver: bridge, name: custom-devops-net } volumes: redis-data-volume: { name: custom-redis-storage }
```

5. Executing Docker Compose & Verifying State

```
$ docker compose -f src/compose/docker-compose.yml up -d
[+] Running 4/4 Network custom-devops-net Created 0.1s Volume custom-redis-storage Created 0.1s Container
redis-cache-container Started 0.6s Container web-app-container Started 1.2s
```

```
$ docker compose -f src/compose/docker-compose.yml ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS redis-cache-container redis:7.2-alpine "docker-entrypoint.s…"
cache-service 12 seconds ago Up 11s (healthy) 0.0.0.0:6379->6379/tcp web-app-container devops-microservice:1.0.0
"gunicorn --bind 0.0…" web-app 11 seconds ago Up 10s (healthy) 0.0.0.0:5000->5000/tcp
```

```
$ curl -s http://localhost:5000/ && curl -s http://localhost:5000/
// Call 1: {"environment":"production","hostname":"web-app-container","status":"success","visitor_count":1} // Call 2
(Visitor count incremented via Redis cache-service):
{"environment":"production","hostname":"web-app-container","status":"success","visitor_count":2}
```

6. Engineering Observations & Conclusion

Key Technical Takeaways

1. **Service Dependency Management:** The service_healthy condition guaranteed that the Web API did not launch until Redis responded to ping requests.
2. **DNS Resolution:** The Web container communicated seamlessly with Redis using the service hostname cache-service.
3. **Declarative Orchestration:** Docker Compose codified the entire multi-tier stack, enabling single-command startup and teardown.